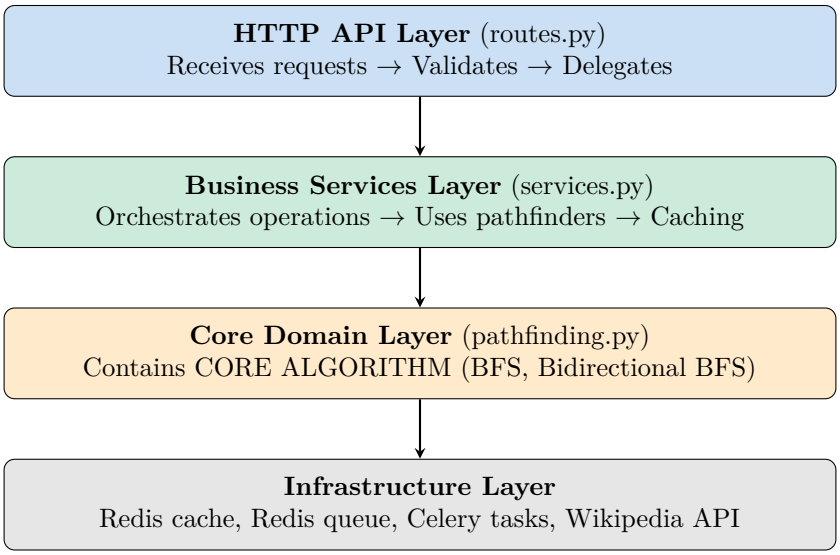
Iris Backend Architecture Guide

Wikipedia Pathfinding Engine



Interview Preparation & Development Reference

Last Updated: January 2026

Contents

1 Project Structure Overview **2**

2 Architecture Layers **2**
2.1 Layer Descriptions . . . 3
2.2 Data Flow Diagram . . . 3

3 Engineering Architecture Decisions **3**
3.1 High-Level Architecture Decisions . . . 3
3.2 Low-Level Implementation Decisions . . . 4
3.3 Scalability Characteristics Summary . . . 6

4 Service Lifecycle & Initialization **6**
4.1 The "When": Lazy Initialization . . . 7
4.2 Component Lifecycle Table . . . 7
4.3 Initialization Flow . . . 7

5 Progress Callback System **7**
5.1 The Problem . . . 7
5.2 The Solution: Callback Injection . . . 7
5.3 Code Flow . . . 8
5.3.1 Step 1: Task Defines the Callback . . . 8
5.3.2 Step 2: Factory Passes Callback to PathFinder . . . 8
5.3.3 Step 3: BFS Passes a Closure into the Wikipedia Fetch . . . 8
5.3.4 Step 3b: Link-Processing Callbacks Close the UI Gap . . . 10
5.3.5 Step 4: Client Sees Progress via Polling . . . 11
5.4 Architecture Diagram: Callback Flow . . . 11
5.5 Why This Design? . . . 11

6 Dependency Injection (DI) Explained **11**
6.1 The Pattern: Singleton + Factory . . . 12
6.2 Swapping Implementations . . . 12

7 File-by-File Breakdown **12**
7.1 Core Files . . . 12
7.2 Infrastructure Files . . . 13

8 Caching Strategy **13**

9 Interview Q&A Cheat Sheet **13**

10 Quick Reference: Where to Make Changes **15**

11 API Endpoints Reference **15**

12 Configuration Reference **15**

1 Project Structure Overview

The Iris backend follows a **Clean Architecture** pattern with clear separation of concerns.

```
iris-web-backend/
|-- app/                          # Main application package
|   |-- __init__.py              # Flask app factory & Celery setup
|   |-- api/                     # HTTP API layer
|   |   |-- routes.py            # API endpoints (Flask blueprints)
|   |   |-- middleware.py        # Decorators: error handling, logging, CORS
|   |   |-- schemas.py           # Marshmallow schemas for validation
|   |   |-- swagger.py           # Flasgger config + API metadata (paths from docstrings)
|   |-- core/                    # Core business logic (THE BRAIN)
|   |   |-- factory.py           # Dependency Injection (ServiceFactory)
|   |   |-- interfaces.py        # Abstract interfaces (contracts)
|   |   |-- models.py            # Data models (dataclasses)
|   |   |-- pathfinding.py       # CORE ALGORITHM (BFS)
|   |   |-- services.py          # Business services orchestration
|   |-- external/                # External API integrations
|   |   |-- wikipedia.py         # Wikipedia API client
|   |-- infrastructure/          # Infrastructure concerns
|   |   |-- cache.py             # Redis cache implementation
|   |   |-- redis_queue.py       # Redis queue for BFS state
|   |   |-- tasks.py             # Celery background tasks
|   |-- utils/                   # Utilities
|       |-- exceptions.py        # Custom exception hierarchy
|       |-- logging.py           # Logging configuration
|-- config/                      # Configuration classes
|-- tests/                       # Test suites
|-- run.py                       # Application entry point
|-- celery_worker.py             # Celery worker entry point
```

Listing 1: Project Directory Structure

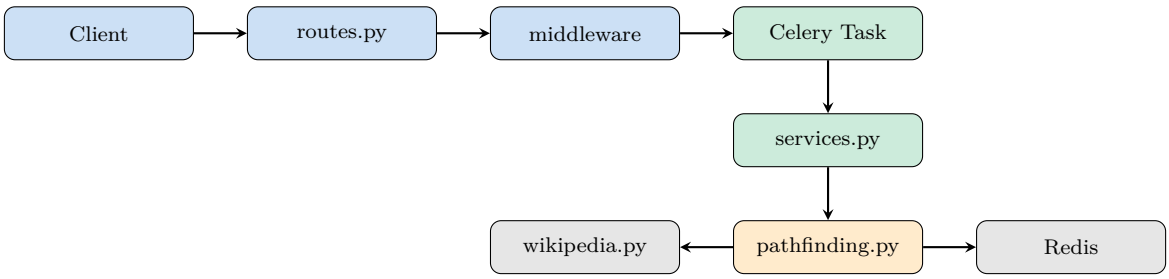
2 Architecture Layers

The codebase follows a **layered architecture** similar to Clean Architecture. Each layer has a single responsibility.

2.1 Layer Descriptions

Layer		Responsibility
HTTP API Layer	`app/api/`	Receives HTTP requests, validates input using Marshmallow schemas, applies cross-cutting middleware (CORS, logging, error handling), delegates to business services, and formats JSON responses. Swagger spec is auto-assembled from YAML docstrings on each route function via Flasgger.
Business Services	`app/core/services.py`	Orchestrates business operations. Checks cache before invoking pathfinding, coordinates between pathfinders and Wikipedia client, encapsulates use cases like "find path" or "explore page."
Core Domain	`app/core/pathfinding.py`	Contains the pure BFS algorithm. No knowledge of HTTP, Redis, or Flask. Only knows about abstract interfaces. This is where the graph traversal logic lives.
Infrastructure	`app/infrastructure/`	Concrete implementations: Redis cache, Redis queue for BFS state, Celery tasks for background processing, Wikipedia API client. All "how" details live here.

2.2 Data Flow Diagram



3 Engineering Architecture Decisions

This section justifies key technical decisions from a **distributed systems** and **scalability** perspective. These are the answers that demonstrate engineering depth.

3.1 High-Level Architecture Decisions

Decision: Asynchronous Task Processing with Celery

Problem: Wikipedia pathfinding can take 30 seconds to 5+ minutes. HTTP requests would timeout.
Decision: Offload pathfinding to Celery background workers. API returns a task ID immediately; client polls for results.
Why This Scales:

- **Horizontal Scaling:** Spin up N Celery workers to handle N concurrent searches.
- **Decoupling:** Web servers handle HTTP; workers handle compute. Each scales independently.
- **Resilience:** If a worker crashes, the task stays in Redis and another worker picks it up.

Trade-off: Added complexity of polling. Considered WebSockets but polling is simpler to

implement and debug.

Decision: Redis as Cache

Problem: Need caching, a job queue, and result storage. Using 3 different systems increases operational overhead.
Decision: Use Redis for all three: Wikipedia link cache, Celery broker, and Celery result backend.
Why This Scales:
- **Single Point of Expertise:** Operations team only needs to master one system.
- **Redis Cluster:** Can scale to hundreds of thousands of operations/second.
- **Memory Efficiency:** Redis is optimized for this exact use case.

Trade-off: Redis is a single point of failure. Mitigated with Redis Sentinel or Redis Cluster in production.

Decision: Stateless Web Servers

Problem: Need to scale web tier horizontally behind a load balancer.
Decision: All application state lives in Redis. Flask servers are completely stateless.
Why This Scales:
- **Instant Horizontal Scaling:** Add/remove web servers without session migration.
- **Zero Downtime Deploys:** Rolling restarts don't lose user state.
- **Container-Friendly:** Perfect for Kubernetes/Docker deployments.

Decision: Separation of API and Worker Deployments

Problem: API servers need to respond quickly; workers need CPU for graph traversal.
Decision: Deploy API servers and Celery workers as separate services with different resource profiles.
Why This Scales:
- **Independent Scaling:** Scale workers based on queue depth; scale API based on request rate.
- **Resource Isolation:** Workers can use 100% CPU without affecting API latency.
- **Cost Optimization:** Use different instance types (API: network-optimized; Workers: compute-optimized).

3.2 Low-Level Implementation Decisions

Decision: Redis-Based BFS Queue with Batch Node Processing

Problem: BFS at depth 5+ can involve millions of nodes. Python's `deque` would exhaust worker memory. Fetching links one node at a time also wastes Wikipedia API capacity.
Decision: Store BFS queue and visited set in Redis. Each iteration pops a **batch** of nodes (`BFS_BATCH_SIZE`, default 50) and fetches all their links in a single `get_links_bulk` call.
Engineering Justification:
- **Memory Safety:** Worker memory stays constant regardless of search depth.
- **Throughput:** Popping 50 nodes and dispatching 50 parallel Wikipedia API calls replaces 50 sequential round-trips per BFS level.
- **Observability:** Can inspect queue depth via Redis CLI for debugging.
- **Crash Recovery:** If worker dies, search state persists in Redis (though we restart fresh currently).

Trade-off: Each Redis `pop_batch` is a network hop, but it is amortised across all nodes in the batch. The net effect is far fewer total round-trips versus per-node fetching.

Decision: Connection Pooling via Singleton Pattern

Problem: Creating new TCP connections to Redis/Wikipedia on every request is expensive.
Decision: Use lazy-loaded singletons for `RedisClient` and `WikipediaClient`.
Engineering Justification:

- **Redis:** Uses `redis.ConnectionPool` – amortizes TCP handshake across requests.
- **Wikipedia:** Uses `requests.Session` – enables HTTP Keep-Alive and connection reuse.
- **TLS Overhead:** Wikipedia uses HTTPS. Reusing connections avoids repeated TLS handshakes (100-300ms saved per request).

Decision: Wikipedia API Fetching: One Request Per Page

Problem: Wikipedia's `prop=links` API returns at most 500 links per request (`pllimit=max`). This cap is **global per API call**, not per title. Grouping multiple titles in one request causes link starvation: a high-link-count hub page (e.g. *Microsoft* with 500+ links) consumes the entire 500-link quota, leaving other pages in the same request with 0 links returned.
Decision: Each page title is fetched in its own API call. Parallelism is preserved via `ThreadPoolExecutor` (`WIKIPEDIA_MAX_WORKERS`, default: 10), so a BFS batch of 50 nodes still issues all 50 requests concurrently.
Engineering Justification:

- **Correctness:** Every page receives its full 500-link allocation regardless of what other pages are in the same BFS batch.
- **I/O Bound:** Wikipedia API calls are network-bound; threads maximise throughput without the starvation risk of multi-title requests.
- **Rate Limiting:** Thread pool size implicitly caps concurrent requests to Wikipedia.

Known Limitation: Wikipedia paginates beyond 500 links via a `continue` token. This implementation does **not** follow pagination; only the first 500 outgoing links of any page are explored. For most articles this is sufficient, but high-connectivity hub pages may have paths that pass through links beyond the first 500. This is a conscious trade-off: following pagination would multiply API calls for hub pages and add latency for marginal accuracy gain.

Decision: Why Bidirectional BFS Was Abandoned

Problem: Bidirectional BFS works by expanding two frontiers simultaneously — one from the start and one from the end — and halting when they meet. This can reduce search space from $O(b^d)$ to $O(b^{d/2})$ in uniform graphs.
Why It Fails for Wikipedia:

- Wikipedia link graphs are **highly asymmetric**. A page like *Adolf Hitler* has thousands of **incoming** links but only hundreds of **outgoing** links.
- The reverse frontier (expanding backward from the target via incoming links) requires fetching "what links here" — a separate Wikipedia API call (`prop=linkshere`) that returns far larger result sets.
- The reverse frontier explodes in size immediately, eliminating any search space reduction and typically making the search **slower** than standard BFS.

Decision: Retain only forward BFS. The `BidirectionalBFSPathFinder` class exists in the codebase and is selectable via the API `algorithm` parameter, but it delegates directly to

`RedisBasedBFSPathFinder`. The `algorithm` field is kept in the API for forward compatibility.
Trade-off: Forward BFS with Wikipedia's dense link graph typically finds paths of length 3–5 within seconds. The asymmetry that breaks bidirectional BFS is also what makes forward BFS fast.

Decision: Cache TTLs Tuned to Data Volatility

Problem: Wikipedia pages change, but not frequently. Need to balance freshness vs. cache hit rate.
Decision: Different TTLs for different data types.
Engineering Justification:
- `wiki_links:*` – 24 hours. Page links rarely change; high reuse value.
- `path:*:*` – 1 hour. Paths are computed results; shorter TTL ensures freshness.
- `bfs_*` – 1 hour with explicit cleanup. Ephemeral search state; cleaned after use.

Decision: UUID-Based Session Isolation for BFS State

Problem: Multiple concurrent searches must not interfere with each other's BFS state.
Decision: Each search generates a UUID. All Redis keys are prefixed: `bfs_queue:<uuid>`, `bfs_visited:<uuid>:*`.
Engineering Justification:
- **No Locking Required:** Each search operates on its own keyspace.
- **Easy Cleanup:** Pattern-based deletion: `DEL bfs_*:<uuid>:*`.
- **Debugging:** Can trace a specific search by its UUID in logs and Redis.

Decision: Dependency Injection via Factory Pattern

Problem: Need testable code without complex DI frameworks.
Decision: Hand-rolled `ServiceFactory` with constructor injection.
Engineering Justification:
- **Testability:** Tests inject mocks directly via constructor.
- **No Magic:** Unlike `@inject` decorators, the wiring is explicit and traceable.
- **Cleanup Method:** `ServiceFactory.cleanup()` resets singletons between tests.

3.3 Scalability Characteristics Summary

Dimension	How It Scales
Concurrent Searches	Add more Celery workers (horizontal).
API Request Rate	Add more Flask/Gunicorn instances behind load balancer.
Cache Size	Redis Cluster with sharding or increase instance memory.
Wikipedia Rate Limits	Configurable thread pool; add delay; use multiple API keys.
Storage (Results)	Redis with TTL-based eviction; results auto-expire after 1 hour.

4 Service Lifecycle & Initialization

Understanding precisely **when**, **where**, and **how** services are initialized demonstrates production awareness.

4.1 The "When": Lazy Initialization

Services are **not** initialized when you run `python run.py`. They are created **on first use**.

- **Flask Process:** First API request triggers service initialization.
- **Celery Worker:** First task pickup triggers service initialization.

4.2 Component Lifecycle Table

Component	Lifecycle	Reasoning
Redis Client	Singleton	Maintains `ConnectionPool`; avoids TCP handshake overhead per request.
Wikipedia Client	Singleton	Holds `requests.Session`; enables HTTP Keep-Alive and TLS session reuse.
Cache Service	Singleton	Shared serialization logic wrapping the Redis client.
Pathfinding Service	Transient	New instance per task. Allows request-specific `progress_callback` and algorithm.

4.3 Initialization Flow

When a Celery task starts:

1. `find_path_task` starts in `infrastructure/tasks.py`.
2. Calls `get_pathfinding_service(algorithm, progress_callback)`.
3. `ServiceFactory` checks `_wikipedia_client`. It's `None` → creates it.
4. To create Wikipedia client, needs `_cache_service`. It's `None` → creates it.
5. To create cache, needs `_redis_client`. It's `None` → creates it.
6. Dependencies wired together via constructor injection; `PathFindingService` returned.
7. **Next task:** Singletons already exist → instant return.

Important

Singletons are Per-Process. If Gunicorn runs 4 workers, there are 4 independent Wikipedia Clients. They share state via the common Redis database, not via shared memory.

5 Progress Callback System

The progress callback allows clients to see **live search progress** rather than waiting blindly.

5.1 The Problem

A BFS search can take minutes. Without progress updates, the client has no idea:

- Is the search still running or stuck?
- How deep has it gone?
- How many nodes has it explored?

5.2 The Solution: Callback Injection

1. **Task creates a callback function** that updates Celery task state.
2. **Callback is injected** into `PathFindingService` via constructor.

3. **BFS passes an `on_page_fetched` closure** into `get_links_bulk`.
4. **Wikipedia client fires the closure** from worker threads as each API response arrives.
5. **Client polls `/tasks/status/<id>`** and sees real-time progress spread across the fetch window.

5.3 Code Flow

5.3.1 Step 1: Task Defines the Callback

```
@celery.task(bind=True)
def find_path_task(self, start_page, end_page, algorithm="bfs"):

    # Define a closure that captures 'self' (the Celery task)
    def progress_update(progress_data):
        self.update_state(
            state="PROGRESS",
            meta=progress_data  # Contains nodes_explored, current_depth, etc.
        )

    # Pass the callback to the service factory
    pathfinding_service = get_pathfinding_service(
        algorithm=algorithm,
        progress_callback=progress_update  # <-- Injected here
    )

    result = pathfinding_service.find_path(search_request)
```

Listing 2: tasks.py – Creating the Progress Callback

5.3.2 Step 2: Factory Passes Callback to PathFinder

```
@classmethod
def create_pathfinding_service(cls, algorithm="bfs", progress_callback=None):
    # ... get singletons ...

    path_finder = RedisBasedBFSPathFinder(
        wikipedia_client,
        cache_service,
        queue_service,
        max_depth,
        batch_size,
        progress_callback  # <-- Passed to the algorithm
    )

    return PathFindingService(path_finder, cache_service, wikipedia_client)
```

Listing 3: factory.py – Injecting the Callback

5.3.3 Step 3: BFS Passes a Closure into the Wikipedia Fetch

Rather than firing all progress updates before the Wikipedia API call (which bunches them into a tight loop invisible to polling clients), BFS defines an `on_page_fetched` closure and passes it into `get_links_bulk`. The Wikipedia client fires this closure **from worker threads** as each individual HTTP response arrives via `as_completed`, spreading updates naturally across the network round-trip window.

```
def _perform_bfs_search(self, start_page, end_page, ...):
    nodes_explored = 0
    nodes_lock = threading.Lock()  # guards against concurrent increments

    while self.queue_service.length(queue_key) > 0:
```

```python
        batch_items = self.queue_service.pop_batch(queue_key, self.batch_size)
        current_depth = batch_items[0]["depth"]

        # Pre-declare so the type-checker sees Callable | None across branches.
        on_page_fetched: Callable[[str, list[str]], None] | None = None
        if self.progress_callback:

            def _on_page_fetched(
                title: str,
                _links: list[str],
                *,
                _d: int = current_depth,  # default arg binds value at def time
            ) -> None:
                nonlocal nodes_explored
                with nodes_lock:
                    nodes_explored += 1
                    count = nodes_explored
                self.progress_callback({
                    "status": "Searching...",
                    "search_stats": {
                        "nodes_explored": count,
                        "current_depth": _d,
                        "last_node": title,
                        # NOTE: stale within batch -- see note below
                        "queue_size": self.queue_service.length(queue_key),
                    },
                    "search_time_elapsed": round(time.time() - start_time, 2),
                })

            on_page_fetched = _on_page_fetched

        # on_page_fetched fires inside the thread pool as each page resolves
        links_bulk = self.wikipedia_client.get_links_bulk(
            [item["page"] for item in batch_items], on_page_fetched
        )
        # ... process each page's links ...
```

Listing 4: pathfinding.py – on_page_fetched Closure

```python
def _fetch_from_wikipedia(self, page_titles, on_page_fetched=None):
    results = {}
    with ThreadPoolExecutor(max_workers=self.max_workers) as executor:
        future_to_title = {
            executor.submit(self._fetch_single_page, title): title
            for title in page_titles
        }
        for future in as_completed(future_to_title):
            title = future_to_title[future]
            page_result = future.result()
            results.update(page_result)
            if on_page_fetched:
                on_page_fetched(title, page_result.get(title, []))
    return results
```

Listing 5: wikipedia.py – Firing the Callback via as_completed

Why this works: With `executor.map` all results are collected before returning, so callbacks would still bunch together. `as_completed` yields each future the moment its HTTP response arrives — progress updates are spread across the ~500–2000 ms fetch window, giving the polling client genuinely smooth feedback.

Thread safety: `on_page_fetched` is called concurrently from multiple worker threads. `nodes_lock` (a `threading.Lock`) ensures `nodes_explored` is incremented atomically. `update_state` writes to Redis, which is inherently thread-safe.

Loop variable capture (B023): Python closures capture variables by reference, not by value. If `current_depth` were referenced as a free variable inside the closure, all threads would see whatever value it holds when they *execute*, not when the closure was *defined* — a classic late-binding bug. Binding it as a keyword-only default argument (`_d=current_depth`) forces evaluation at definition time, making each iteration's closure hold its own snapshot of the depth.

Queue size during the fetch phase is stale. `on_page_fetched` fires *during* the Wikipedia fetch, before the link-processing loop has pushed any new nodes. The `queue_size` at that point reflects the queue after `pop_batch` removed the current batch but before any new discoveries are enqueued — it will read lower than reality until the next batch starts.

5.3.4 Step 3b: Link-Processing Callbacks Close the UI Gap

After `get_links_bulk` returns, the link-processing loop runs: for each of the 50 pages, it checks the visited set, stores new paths in Redis, and pushes newly discovered nodes onto the queue. This loop is entirely Redis round-trips and can take several seconds for pages with hundreds of links. No Wikipedia fetch is in flight during this time, so without intervention the client would stall — polling every second and seeing unchanged values.

To prevent this, `progress_callback` is fired once per page *after* its links have been enqueued. At that point `queue_size` reflects the newly added nodes, so the client sees the queue growing in real time:

```python
for item in batch_items:
    current_page = item["page"]
    links = links_bulk.get(current_page, [])
    current_path = self.cache_service.get(f"{paths_key}:{current_page}")

    for link in links:
        if link == end_page:
            return {"path": current_path + [link], ...}
        # check visited, store path, push to queue ...

    # Fire after all links for this page are enqueued.
    # queue_size now reflects the newly added nodes.
    if self.progress_callback:
        self.progress_callback({
            "status": "Searching...",
            "search_stats": {
                "nodes_explored": nodes_explored,
                "current_depth": current_depth,
                "last_node": current_page,
                "queue_size": self.queue_service.length(queue_key),
            },
            "search_time_elapsed": round(time.time() - start_time, 2),
        })
```

Listing 6: pathfinding.py – Link-Processing Callbacks

Combined update rate per batch:

- **During Wikipedia fetch:** up to 50 callbacks fired from worker threads as HTTP responses arrive via `as_completed`.
- **During link processing:** 50 callbacks fired sequentially, one per page, as its links are enqueued.
- **Total:** up to 100 `update_state` Redis writes per batch. With the client polling every 1 s and each batch taking 2–8 s, the client observes meaningfully different state on every poll throughout the entire batch.

Redis cost is negligible. Each callback is 1 `SET` (`update_state`) + 1 `LLEN` (`queue_size`) = 2 ops. 100 callbacks = 200 extra ops per batch. The link-processing loop itself already performs

~60 000 Redis ops per batch (visited checks, path stores, queue pushes). The callbacks add <0.4% overhead regardless of the number of concurrent searches.

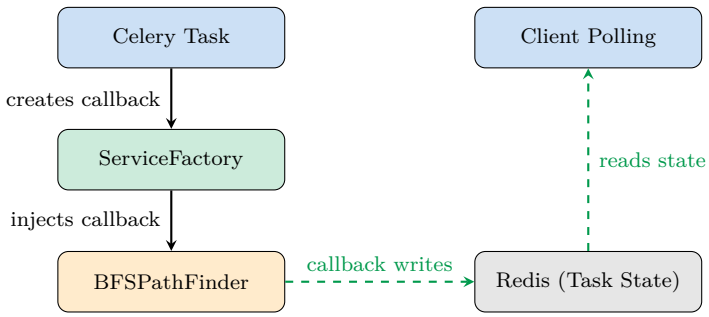
5.3.5 Step 4: Client Sees Progress via Polling

```
GET /tasks/status/abc-123

{
    "status": "IN_PROGRESS",
    "task_id": "abc-123",
    "progress": {
        "status": "Searching...",
        "search_stats": {
            "nodes_explored": 42,
            "current_depth": 3,
            "last_node": "Machine learning",
            "queue_size": 150
        },
        "search_time_elapsed": 5.2
    }
}
```

Listing 7: Client Polling Response

5.4 Architecture Diagram: Callback Flow



5.5 Why This Design?

Benefit	Explanation
Decoupling	The BFS algorithm doesn't know about Celery. It just calls a function.
Testability	In tests, pass a mock callback that records calls instead of updating Celery.
Flexibility	Easy to change reporting frequency or add more metrics without touching BFS.
No Polling Inside Worker	Worker doesn't poll; it pushes. Client pulls on its own schedule.

6 Dependency Injection (DI) Explained

Note

Location: `app/core/factory.py` – `ServiceFactory` class

6.1 The Pattern: Singleton + Factory

```python
class ServiceFactory:
    _redis_client = None      # Singleton storage
    _cache_service = None
    _wikipedia_client = None

    @classmethod
    def get_cache_service(cls):
        if cls._cache_service is None:
            redis_client = cls.get_redis_client()
            cls._cache_service = RedisCache(redis_client)
        return cls._cache_service

    @classmethod
    def create_pathfinding_service(cls, algorithm="bfs", progress_callback=None):
        # Retrieve singletons, then inject into new instance
        wiki = cls.get_wikipedia_client()
        cache = cls.get_cache_service()
        queue = cls.get_queue_service()

        path_finder = RedisBasedBFSPathFinder(wiki, cache, queue,
    progress_callback)
        return PathFindingService(path_finder, cache, wiki)
```

Listing 8: ServiceFactory Implementation

6.2 Swapping Implementations

To use Memcached instead of Redis:

1. Create `MemcachedCache` implementing `CacheServiceInterface`.
2. Modify `ServiceFactory.get_cache_service()` to return `MemcachedCache`.
3. Done. All services automatically use the new cache.

7 File-by-File Breakdown

7.1 Core Files

File	Purpose
`app/core/pathfinding.py`	**CORE ALGORITHM**. Redis-based BFS. Modify for algorithm changes.
`app/core/services.py`	Business orchestration. Cache checking, validation, timing.
`app/core/factory.py`	DI container. Manages singletons and object creation.
`app/core/interfaces.py`	Abstract contracts for testability and loose coupling.
`app/core/models.py`	Dataclasses for requests, results, and DTOs.

7.2 Infrastructure Files

File	Purpose
`app/infrastructure/cache.py`	Redis cache implementation with connection pooling.
`app/infrastructure/redis_queue.py`	FIFO queue for BFS state (push, pop, length).
`app/infrastructure/tasks.py`	Celery task definitions with retry logic and progress updates.
`app/external/wikipedia.py`	Wikipedia API client with bulk fetching and redirect handling.

8 Caching Strategy

Key Pattern	TTL	Purpose
`wiki_links:{page}`	24h	Caches Wikipedia page links. High reuse.
`path:{start}:{end}`	1h	Caches completed path results.
`bfs_visited:<uuid>:*`	1h	Temporary visited set. Cleaned after search.
`bfs_paths:<uuid>:*`	1h	Temporary path tracking. Cleaned after search.
`bfs_queue:<uuid>`	Session	BFS queue. Explicitly deleted after search.

9 Interview Q&A Cheat Sheet

Q: If you had to modify the core pathfinding algorithm

Answer: `app/core/pathfinding.py` – specifically the `RedisBasedBFSPathFinder` class and its `find_path()` method.

Q: When and where do services get initialized?

Answer: Services are **lazily initialized** by `ServiceFactory` (`app/core/factory.py`) on first use. Not at app startup. Flask: first request. Celery: first task.

Q: Which objects are reused vs. created fresh?

Answer:
- **Reused (Singletons):** Redis Client, Wikipedia Client, Cache Service. Maintains connection pools.
- **Fresh (Transient):** PathFindingService. New per task for isolated callbacks.

Q: How does the progress callback work?

Answer:
1. Celery task defines a closure that calls `self.update_state()`.
2. This closure is passed to `ServiceFactory.create_pathfinding_service()`.
3. Factory injects it into `RedisBasedBFSPathFinder` constructor.
4. BFS calls the callback every N nodes, writing progress to Redis.
5. Client polls `/tasks/status/<id>` to read progress.

Q: Why use Celery instead of handling pathfinding synchronously?

Answer: Pathfinding can take minutes. Synchronous handling would:
- Timeout HTTP connections (typically 30-60s limit).
- Block web server threads, reducing throughput.
- Provide no progress visibility to users.

Celery enables horizontal scaling, resilience, and real-time progress.

Q: Why Redis for BFS queue instead of Python deque?

Answer: Memory safety. BFS at depth 5+ can involve millions of nodes. Redis:
- Offloads memory from worker process.
- Enables inspection via Redis CLI.
- Provides crash-recovery potential (state persists).

Trade-off: Network latency per operation (mitigated by batching).

Q: How does the system scale horizontally?

Answer:
- **API:** Stateless Flask servers behind load balancer.
- **Workers:** Add Celery workers to increase search throughput.
- **Cache:** Redis Cluster for sharding and replication.

Q: Explain singletons across processes.

Answer: Singletons are per-process. 4 Gunicorn workers = 4 Wikipedia Clients. They share state via Redis (the database is the shared memory), not in-process memory.

Q: Why is the Wikipedia Client a singleton?

Answer: It holds a `requests.Session` which provides:
- HTTP Keep-Alive (connection reuse).
- TLS session resumption (avoids 100-300ms handshake).
- Cookie persistence if needed.

Creating new clients per request would destroy these benefits.

Q: How would you add a new pathfinding algorithm?

Answer:
1. Create class implementing `PathFinderInterface` in `pathfinding.py`.
2. Add case in `ServiceFactory.create_pathfinding_service()`.
3. Update API schema to accept new algorithm name.
4. Add tests in `tests/integration/test_pathfinding.py`.

10 Quick Reference: Where to Make Changes

Goal	Location
Modify BFS algorithm	`app/core/pathfinding.py`
Add new API endpoint	`app/api/routes.py`
Change request validation	`app/api/schemas.py`
Add new Celery task	`app/infrastructure/tasks.py`
Swap cache implementation	Implement interface, update `factory.py`
Add new exception type	`app/utils/exceptions.py`
Change configuration	`config/base.py`
Modify Wikipedia API logic	`app/external/wikipedia.py`

11 API Endpoints Reference

Method	Endpoint	Description
POST	`/getPath`	Start pathfinding. Returns task ID.
GET	`/tasks/status/<id>`	Poll for task status and progress.
POST	`/explore`	Explore links from a page.
GET	`/health`	System health check.
GET	`/api`	API information.
POST	`/cache/clear`	Admin: clear cache by pattern.
GET	`/api/docs`	Swagger UI (interactive API documentation).

12 Configuration Reference

Variable	Default	Description
`REDIS_URL`	localhost:6379	Redis connection URL.
`MAX_SEARCH_DEPTH`	6	Maximum BFS depth.
`BFS_BATCH_SIZE`	50	Pages per batch in BFS.
`CACHE_TTL`	86400	Default cache TTL (24h).
`WIKIPEDIA_MAX_WORKERS`	10	Thread pool for Wikipedia API.
`CELERY_TASK_SOFT_TIME_LIMIT`	300	Soft timeout (5m).
`CELERY_TASK_TIME_LIMIT`	600	Hard timeout (10m).